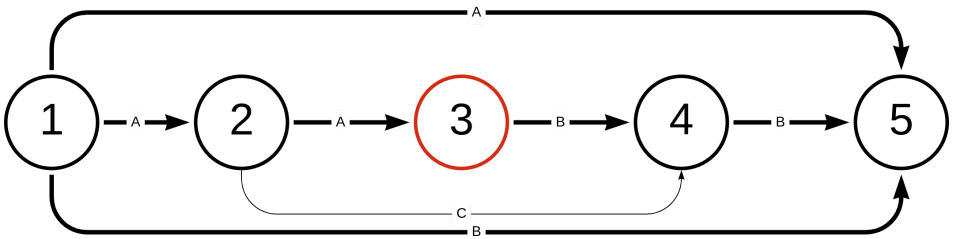
Bidirectional Traversal Framework

A bidirectional traversal consists of two traversals starting from different nodes and returning the paths to where both traversals collide. The Bidirectional Traversal Framework allows the description of such a traversal with the `BidirectionalTraversalDescription`.

The following is a visual representation of a bidirectional traversal from start node `1` and end node `5`, where all bold relationships are in the result paths given the following restrictions:

- The start side traverser only traverses relationships of type `A`.
- The end side traverser only traverses relationships of type `B`.



Note that paths where only one of the traversers reaches the opposite start/end node are also included, such as `(1)-[:A]→(5)` and `(1)-[:B]→(5)`. In the accepted path `(1)-[:A]→(2)-[:A]→(3)-[:B]→(4)-[:B]→(5)`, the traversers collide in the middle at node `3`.

NOTE

The `Uniqueness` is shared between both traversals, which means that there will be no results if the `Uniqueness` is set to `Uniqueness.NODE_GLOBAL` (which is the default) as both traversals need to reach the same node to cause a collision. Therefore, when using `BidirectionalTraversalDescription`, always set the `Uniqueness` manually. For more information on the available options, see `Uniqueness` options.

Defining the bidirectional traverser

When creating `BidirectionalTraversalDescription`, it is possible to choose the `TraversalDescription` that each side takes. This can be done by setting it individually for each side (`startSide` / `endSide`) or setting one for both (`mirroredSides`).

AI search

Start and end side traversal

The following is an example of `BidirectionalTraversalDescription` with a separate traverser for the `startSide()` and `endSide()`:

```
BidirectionalTraversalDescription td = transaction
        .bidirectionalTraversalDescription()
        .startSide(transaction
                .traversalDescription()
                .expand(PathExpanders.forTypeAndDirection(RelationshipType.withName("A"),
Direction.OUTGOING))
                .uniqueness(Uniqueness.RELATIONSHIP_GLOBAL))
        .endSide(transaction
                .traversalDescription()
                .expand(PathExpanders.forTypeAndDirection(RelationshipType.withName("B"),
Direction.INCOMING))
                .uniqueness(Uniqueness.RELATIONSHIP_GLOBAL));
td.traverse(startNode, endNode);
```

One side is traversing forward from the start node and expanding all outgoing relationships of Type `A`. The other side is traversing backward from the end node and expanding all incoming relationships of Type `B`.

The final paths go from the start node to the end node, where all relationships have an outgoing direction. Possible paths are:

- All relationships are of Type `A`.
- All relationships are of Type `B`.
- The relationships from the start node are Type `A` and, after the collision, they are all of type `B`.

Mirrored traversal

The `mirroredSides(TraversalDescription)` method sets both the start side and end side of this bidirectional traversal. However, the end side is the reverse of the start. For the `Direction` of `Relationships`, this means that `OUTGOING` becomes `INCOMING` and vice versa. The `PathExpander` interface also comes with a `reverse()` function, which should be overridden if used in a `mirroredSides` implementation.

```
BidirectionalTraversalDescription td = transaction
        .bidirectionalTraversalDescription()
        .mirroredSides(transaction
                        .traversalDescription()
```

```
                                .uniqueness(Uniqueness.RELATIONSHIP_GLOBAL));
td.traverse(startNode, endNode);
```

Side selector

In a bidirectional traversal, the traverser selects which side (start or end) to move on each step. It is possible to change how this works by implementing the `SideSelectorPolicy` interface, as it has a function for determining from which side to traverse the next step.

- `Direction.OUTGOING` — The traverser coming from the start node.
- `Direction.INCOMING` — The traverser coming from the end node.

The built-in policies include:

- `SideSelectorPolicies.LEVEL` — Stops traversal if the combined depth is larger than the given maximum depth.
- `SideSelectorPolicies.LEVEL_STOP_DESCENT_ON_RESULT` — Stops as soon as a result is found.
- `SideSelectorPolicies.ALTERNATING` — Alternates which branch continues the traversal.

The `SideSelectorPolicy` optionally takes `maxDepth`, which represents the combined depth that both sides must adhere to.

The following is an example of how to use the built-in `SideSelectorPolicy` with a max combined depth of 5:

```
BidirectionalTraversalDescription td = transaction
        .bidirectionalTraversalDescription()
        .mirroredSides(transaction
                                .traversalDescription()
                                .uniqueness(Uniqueness.RELATIONSHIP_GLOBAL))
                                .sideSelector(SideSelectorPolicies.LEVEL, 5);
td.traverse(startNode, endNode);
```

Collision policies

`BranchCollisionPolicy` defines when a collision is detected and accepted in a bidirectional traversal. Given an evaluator and a path predicate, `BranchCollisionPolicy` creates `BranchCollisionDetector`, which detects collisions between two traversers and adds the resulting path to the result if it satisfies the conditions given by the collision evaluator and Uniqueness constraints.

These are the built-in `BranchCollisionPolicies`:

- `STANDARD` — Returns all paths with a collision.
- `SHORTEST_PATH` — Returns all paths with the smallest traversal depth.

The following is an example of how to use the built-in `BranchCollisionPolicy`:

```
BidirectionalTraversalDescription td = transaction
        .bidirectionalTraversalDescription()
        .mirroredSides(transaction
                                .traversalDescription()
                                .uniqueness(Uniqueness.RELATIONSHIP_GLOBAL))
        .collisionPolicy(BranchCollisionPolicies.SHORTEST_PATH);
td.traverse(startNode, endNode);
```

Collision evaluator

The convenience method `collisionEvaluator()` adds `PathEvaluator`, which validates paths in a valid collision. Using the method multiple times results in the combination of the evaluators.

```
BidirectionalTraversalDescription td = transaction
    .bidirectionalTraversalDescription()
    .mirroredSides(transaction
       .traversalDescription()
       .uniqueness(Uniqueness.RELATIONSHIP_GLOBAL))
    .collisionEvaluator(Evaluators.atDepth(3));
td.traverse(startNode, endNode);
```

Prev
‹ Traversal Framework Java API

Next
Traversal Framework examples ›

Contents

Defining the bidirectional traverser
Start and end side traversal
Mirrored traversal
Side selector
Collision policies
Collision evaluator

LEARN

Sandbox
Neo4j Community Site
Neo4j Developer Blog
Neo4j Videos
GraphAcademy
Neo4j Labs

SOCIAL

Twitter
Meetups
Github
Stack Overflow

Want to Speak?

CONTACT US →

US: 1-855-636-4532
Sweden +46 171 480 113
UK: +44 20 3868 3223
France: +33 (0) 1 88 46 13 20

© 2026 Neo4j, Inc.
Terms | Privacy | Sitemap

Neo4j®, Neo Technology®, Cypher®, Neo4j® Bloom™ and Neo4j® Aura™ are registered trademarks of Neo4j, Inc. All other marks are owned by their respective companies.